



Bilan de gestion d'équipe et de projet

Projet Genie Logiciel - gl16

ENSIMAG 2025/2026

AIT MOHAMED Younes (aitmohay)

BEAURAIN Tom (beurato)

BEN ISMAIL Wajd (benismaw)

FURLANI DA SILVA SOARES Manuela (furlanim)

MARIA ENGLER PINTO Carlos (mariaenc)

1 Contexte et objectif du bilan

Ce document présente un bilan a posteriori de la gestion d'équipe et du déroulement du projet GL autour du compilateur **decac**. L'objectif n'est pas d'évaluer la qualité intrinsèque de notre démarche, mais l'analyse que nous en faisons après coup : comment nous avons organisé le travail, comment nous avons séquencé la conception et le développement des étapes B (vérifications contextuelles) et C (génération de code), et comment l'effort s'est réparti entre analyse, conception, codage, validation et documentation. Les dates figurant dans nos supports de planification n'ayant pas été mises à jour tout au long du projet, nous nous concentrons volontairement sur la structure du travail et sur les choix organisationnels concrets (découpage en lots, rotation des rôles, intégration, validation).

2 Méthode de développement et organisation de l'équipe

Dès le départ, nous nous sommes appuyés sur une charte de travail commune visant à garantir une organisation efficace, une communication claire et un cadre partagé (qualité, respect des délais, gestion des désaccords). Cette charte servait de référence en cas de blocage et a influencé plusieurs décisions concrètes : privilégier la stabilité de la branche principale, expliciter les responsabilités par tâche, et instaurer une rotation des rôles afin que chacun acquière une expérience technique sur l'ensemble de la chaîne de compilation.

Sur la méthode de développement, nous avons suivi une démarche incrémentale, structurée en cycles courts : pour chaque fonctionnalité, nous commençons par une analyse ciblée de la spécification, puis nous fixons les choix d'interface et de conception. Ensuite, nous implémentons la fonctionnalité sur une branche dédiée, avant d'en valider le comportement par des tests (syntaxiques, contextuels et/ou exécution sur IMA) et une relecture. Cette alternance analyse-conception-codage-validation a été répétée tout au long du projet. Elle a permis de livrer une base fonctionnelle, puis de l'étendre progressivement, tout en limitant les régressions.

A posteriori, ce cadre a été particulièrement utile dans les périodes où l'avancement technique dépendait de conventions communes, lorsque plusieurs modules évoluent en parallèle, l'absence de conventions partagées se traduit rapidement par des intégrations coûteuses. Le fait d'avoir une charte a simplifié la remise à plat : retour aux règles décidées collectivement, clarification des interfaces, et décision plus rapide.

3 Découpage du projet (aligné avec la planification)

Notre découpage a repris la structure de notre planning : nous avons séparé le projet en blocs techniques correspondant aux dépendances naturelles d'un compilateur. Concrètement, nous avons distingué :

- **Génération de code itératif (sans objet)** : lot incrémental couvrant les bases nécessaires à une chaîne exécutable sur IMA. Ce bloc regroupait notamment les déclarations, expressions de base, affectations, structures de contrôle, opérations, puis des tests associés.
- **Parser Objets** : ajout des constructions de classes (déclarations de classes, méthodes, attributs), avec une attention particulière aux priorités (cast, sélection . ,

appels chaînés) et à la stabilité de l'AST produit.

- **Analyse contextuelle Objets (étape B objet)** : héritage, redéfinitions, visibilité, typage des appels, cohérence globale, avec tests associés.
- **Génération de code Objets (étape C objet)** : allocation des objets, construction et utilisation des tables de méthodes (vtables), appels dynamiques.
- **Extension ARM** : Implémentation du compilateur pour une machine ARM.
- **Travaux transverses** : options de `decac`, scripts de test, consolidation de la documentation.

Chaque étape a été découpée en sous-parties afin d'accélérer et de faciliter le travail en parallèle avec moins de conflits Git : chaque bloc avait une zone de code privilégiée (grammaire/AST, règles contextuelles, conventions d'appel et IMA), tout en conservant une vision claire des dépendances entre lots.

3.1 Analyse critique de la rotation et de la charge de travail

Même si nous avons parfois eu des personnes plus à l'aise sur une partie à un moment donné (par exemple le parser ou le codegen), nous avons volontairement adopté une logique *rotative* : les rôles ont changé au cours du projet, afin que chacun développe des compétences sur plusieurs étages du compilateur, et afin d'éviter un point unique de défaillance.

Dans la pratique, cette rotation s'est traduite par :

- des passages de relais planifiés sur les lots (finir une fonctionnalité et transmettre conventions + tests) ;
- des relectures croisées systématiques (une personne n'ayant pas écrit le code vérifie l'intégration et les tests) ;
- une alternance entre développement et validation (écrire une fonctionnalité, puis écrire/renforcer les tests d'une autre).

Avec recul, l'intérêt principal de la rotation des rôles a été la compréhension globale du projet.

Cependant, nous devons admettre une limite majeure à ce système : la répartition inégale de la charge de travail. Si la rotation a fonctionné sur les étapes initiales, la complexité de la partie Objet a rompu cet équilibre.

Le développement du Codegen Objet a demandé un investissement temporel et technique bien plus important que prévu (gestion du tas, appels dynamiques, héritage complexe). Pour respecter les délais, 2 personnes ont dû prendre en charge la quasi-totalité de cette partie, mettant fin de fait à la rotation sur le cœur du compilateur. Ce coût a été double : une surcharge de travail et une perte de visibilité technique pour le reste de l'équipe sur la phase finale du projet.

4 Historique du projet : ordre réel des étapes B et C

4.1 Pourquoi consolider d'abord le socle sans objet

Nous avons commencé par effectuer les tests pour les étapes A et B avant d'implémenter la partie avec objet. Ce travail a permis de valider la structure du compilateur

et d'assurer la stabilité des bases du projet. Ensuite, nous avons continué avec l'implémentation des tests et de la génération de code pour l'étape C. Cependant, en raison de contraintes de temps, nous n'avons pas pu ajouter plus de tests après la finalisation de l'implémentation.

4.2 Étape B objet : progression par dépendances sémantiques

L'étape B (vérifications contextuelles) a été abordée selon une logique proche d'une vérification en plusieurs passages, car le mode objet impose des dépendances fortes :

- **Structure des classes et hiérarchie** : connaître toutes les classes et leurs super-classes, vérifier l'héritage (et éviter des incohérences).
- **Membres (champs / signatures)** : enregistrer les champs et signatures de méthodes, vérifier les redéfinitions et la visibilité.
- **Corps des méthodes** : vérifier les instructions et expressions (`return`, `this`, appels de méthodes, sélection), en s'appuyant sur les informations stabilisées aux étapes précédentes.

Ce séquençement a été essentiel : vérifier trop tôt un corps de méthode revient à raisonner sur un environnement incomplet, ce qui produit soit des rejets incorrects, soit des décorations insuffisantes pour le codegen. C'est l'une des raisons pour lesquelles nous avons veillé à stabiliser l'AST et les définitions (types/classes/membres) avant d'insister sur des cas limites.

4.3 Étape C objet : intégration par mécanismes

Sur la génération de code objet, nous avons suivi une intégration incrémentale, par mécanismes, parce que le débogage est nettement plus difficile lorsqu'un appel dynamique échoue sans test ciblé. Notre progression a été la suivante :

- **Allocation et représentation mémoire** : `new`, mise en place du layout objet (pointeur de vtable + champs), initialisations.
- **Tables des méthodes (vtables)** : construction cohérente avec héritage et redéfinition (slots), vérification par tests de dispatch.
- **Appels dynamiques et convention d'appel** : passage des paramètres, positionnement de `this`, prologue/épilogue, discipline pile et sauvegarde de registres si nécessaire.

Cette stratégie a été guidée par l'observabilité : isoler chaque mécanisme permet de localiser les erreurs et de limiter les effets en cascade (par exemple une erreur d'index méthode dans vtable peut ressembler à une erreur d'allocation si l'on ne dispose pas de tests discriminants).

5 Processus d'intégration, validation et communication

5.1 Workflow Git et intégration

Pour le développement de notre compilateur, nous avons principalement utilisé l'IDE Visual Studio Code, qui nous a permis de travailler efficacement sur les différentes parties du projet, notamment la génération de code et les tests. Nous avons aussi intégré Git pour

la gestion des versions et Maven pour la compilation et l'exécution des tests. Nous avons mis en place une intégration continue avec des branches dédiées aux fonctionnalités, et chaque membre de l'équipe travaillait dans une branche séparée avant de faire une fusion avec la branche principale.

Nous avons adopté un workflow Git basé sur des branches dédiées par fonctionnalité/correction et sur des revues avant intégration. La branche principale a été traitée comme une branche stable : une intégration n'était faite que si la compilation passait et si les tests pertinents ne régressaient pas. Dans les périodes de forte évolution (notamment sur l'objet), nous avons parfois introduit une étape intermédiaire (branche de validation) afin d'éviter d'intégrer trop tôt des changements touchant simultanément grammaire, AST, contexte et codegen.

5.2 Stratégie de tests (validation progressive)

Notre stratégie de validation a suivi :

- **tests de syntaxe** (valid/invalid) pour stabiliser le parser (et éviter de déboguer du contexte sur un AST erroné) ;
- **tests contextuels** pour les règles objet structurantes (héritage, redéfinition, visibilité, typage des appels) ;
- **tests codegen** orientés mécanismes (allocation, vtable, appel dynamique, pression registre et discipline pile).

Nous avons subdivisé chacune de ces catégories en deux sous-parties : sans-objet et objet afin de pouvoir tester indépendamment les paliers d'évolution du compilateur et de lancer des suites de tests ciblées pour identifier rapidement si une modification sur la partie objet entraînait une régression sur le code procédural. Avec recul, c'est la validation qui a consommé la part la plus importante de l'effort, davantage que l'écriture du code lui-même. Les cas qui ont demandé le plus d'itérations sont typiquement ceux où plusieurs niveaux interagissent : priorités du parser (cast/sélection/appels), compatibilités `null` en contexte objet, accès `protected`, et conventions d'appel en présence de polymorphisme.

5.3 Communication et gestion des désaccords

La charte de travail encourageait une communication simple. Dans la pratique, nous avons séparé :

- les échanges rapides pour les blocages immédiats ;
- les discussions techniques traçables via issues / messages liés aux merge requests ;
- réunions physiques : Au moins une fois par semaine pour les points de conception majeurs.
- suivi quotidien : Discussions sur WhatsApp ou Discord pour suivre l'avancement de chacun et débloquer les problèmes rapides.
- la documentation `docs/` pour conserver les décisions de conventions (utile lors des rotations de rôles).

La composition multiculturelle de notre équipe a représenté à la fois un défi et une opportunité. Plusieurs membres n'ayant pas le français comme langue maternelle, nous avons régulièrement dû basculer vers l'anglais lors de nos réunions techniques pour garantir la compréhension de tous.

Bien que ce bilinguisme ait occasionnellement entraîné de rares confusions sur des points techniques précis, le bilan de cette expérience est largement positif :

- **Développement professionnel** : Cette immersion nous a forcés à sortir de notre "bulle" linguistique et à pratiquer l'anglais technique, compétence indispensable pour notre future carrière d'ingénieur.
- **Adaptabilité** : Nous avons dû faire preuve de pédagogie et de patience, en reformulant les concepts complexes de la compilation dans les deux langues pour s'assurer d'un consensus total avant le codage.

Cette dimension internationale, bien que parfois coûteuse en temps de réunion, a renforcé la cohésion du groupe et notre capacité à travailler dans un environnement globalisé.

Lors de désaccords, la démarche a été de revenir à un point vérifiable : un test reproductible, une règle de la spécification, un impact sur la stabilité de la branche principale, puis de décider en privilégiant l'intégration la plus sûre dans le temps imparti.

6 Répartition qualitative de l'effort et analyse critique

Sans suivi horaire strict, notre répartition ressentie de l'effort peut se résumer ainsi :

- **Analyse / compréhension** : forte au début de chaque lot (notamment objet), puis ponctuelle lors des cas limites.
- **Conception** : significative dès que plusieurs modules interagissent (AST/décorations/conventions pile-tas).
- **Codage** : majoritaire en volume, mais rarement la partie la plus coûteuse.
- **Validation / débogage** : très coûteuse, surtout pour l'objet et la génération de code.
- **Documentation** : plutôt consolidée en fin, mais alimentée progressivement par des notes de décision.

A posteriori, notre organisation par blocs a bien fonctionné, et la rotation des rôles a amélioré la compréhension collective. Le point que nous améliorerions concerne la stabilisation plus précoce de certaines conventions transverses (par exemple : forme attendue de l'AST pour sélection/appel, règles exactes de compatibilité/sous-typage, représentation des offsets et index de méthodes). Documenter plus tôt ces conventions aurait réduit certains allers-retours lors des intégrations. Néanmoins, l'approche incrémentale et orientée tests s'est révélée adaptée : elle a limité les régressions et a permis de converger vers un compilateur maintenable malgré la complexité croissante de l'extension objet.

7 Conclusion

En conclusion, notre gestion de projet a combiné un découpage modulaire clair et une progression par fondations, garantissant la stabilité du socle procédural avant l'intégration des mécanismes objets. Cette rigueur technique, appuyée par une stratégie de tests graduelle, nous a permis de livrer un compilateur fonctionnel et maintenable.

Toutefois, ce projet a surtout été une expérience humaine riche en enseignements. Si la charte de travail et la rotation des rôles ont favorisé une compréhension globale du système, la réalité technique nous a confrontés à des limites organisationnelles. La complexité de la phase objet a imposé une spécialisation accrue, créant un déséquilibre

de charge de travail au sein de l'équipe. De même, notre stratégie de validation a révélé l'importance cruciale d'une communication centralisée pour éviter les redondances.

Au final, ce projet nous a permis de sortir de notre "bulle" linguistique et technique. Nous en retirons une leçon fondamentale pour notre future carrière d'ingénieur : la réussite d'un logiciel complexe ne dépend pas seulement de la qualité du code, mais de la capacité de l'équipe à anticiper la répartition de l'effort et à documenter ses conventions pour maintenir une agilité réelle face aux imprévus.